

# Light Class activity: Flask

YKumar, 2024-6-20

This lab sets up a Flask application with SQLAlchemy to manage a to-do list, allowing users to add, update, and delete tasks. The HTML template uses Semantic UI for styling. The routes in the Flask application handle the different actions (home, add, update, delete) and interact with the SQLite database.

## Contents

|                                                                                                                                                                      |    |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 1. Meet the Flask .....                                                                                                                                              | 1  |
| 2. Always Be Coding with ChatGPT .....                                                                                                                               | 2  |
| Step 1: Set Up Flask Environment.....                                                                                                                                | 2  |
| Step 2: Create Flask Application Structure .....                                                                                                                     | 3  |
| Step 3: <code>app.py</code> - Flask Application.....                                                                                                                 | 3  |
| Step 4: <code>templates/base.html</code> - Base Template.....                                                                                                        | 4  |
| Step 5: <code>templates/index.html</code> - Index Template .....                                                                                                     | 4  |
| Step 6: <code>static/style.css</code> - CSS Styles .....                                                                                                             | 5  |
| Step 7: Run the Application .....                                                                                                                                    | 8  |
| 3. Verify with Gemini .....                                                                                                                                          | 11 |
| 4. Add some Claude .....                                                                                                                                             | 16 |
| 5. ChatGPT to the rescue .....                                                                                                                                       | 22 |
| Standard Set Up (if LLMS do not work for you): follow<br><a href="https://www.youtube.com/watch?v=3vfum74ggHE">https://www.youtube.com/watch?v=3vfum74ggHE</a> ..... | 27 |
| Set up the Flask application .....                                                                                                                                   | 27 |
| HTML template (base.html): .....                                                                                                                                     | 29 |
| Set up environmental variables .....                                                                                                                                 | 30 |
| Export environmental variables.....                                                                                                                                  | 30 |
| Run the app.....                                                                                                                                                     | 31 |

## 1. Meet the Flask

Create a folder **Flask**, open it in VSC and open a new terminal

Observe: <https://flask.palletsprojects.com/en/3.0.x/quickstart/> page

Create a file named **hello.py** and run the following code

```
from flask import Flask
```

```
app = Flask(__name__)
@app.route("/")
def hello_world():
    return "<p>Hello, World!</p>"
```

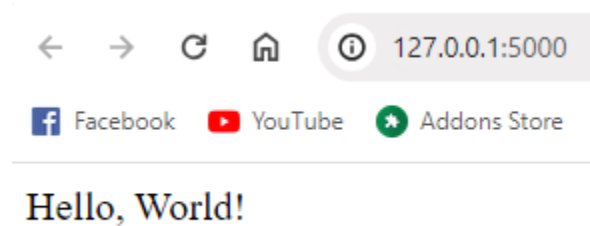
Install flask: **pip3 install flask**

Observe explanation with the top part of the QuickStart tutorial

Run using **flask --app hello run**

Follow the port link: **Running on** <http://127.0.0.1:5000>

**Expected Result:**



## 2. Always Be Coding with ChatGPT

<https://chatgpt.com/>

convert this app to a fancy flask to-do app with navbar and working functionality. Make it a Gryffindor style from flask import Flask

```
app = Flask(__name__)

@app.route("/")
def hello_world():
    return "<p>Hello, World!</p>"
```

ChatGPT can make mistakes. Check important info.

### Output

To create a Gryffindor-themed Flask to-do app with a navbar and working functionality, you can follow the steps below. This involves setting up Flask, creating the necessary HTML templates, and adding some CSS for the Gryffindor theme.

### Step 1: Set Up Flask Environment

First, ensure you have Flask installed. You can install it using pip:

```
bash
Copy code
pip install flask
```

## Step 2: Create Flask Application Structure

Create a directory structure for your Flask app:

```
csharp
Copy code
gryffindor_todo_app/
├── app.py
├── templates/
│   ├── base.html
│   └── index.html
├── static/
│   └── style.css
```

## Step 3: `app.py` - Flask Application

Create `app.py` for your Flask application logic:

```
python
Copy code
from flask import Flask, render_template, request, redirect, url_for

app = Flask(__name__)

# Sample data
todos = []

@app.route('/')
def index():
    return render_template('index.html', todos=todos)

@app.route('/add', methods=['POST'])
def add_todo():
    todo = request.form.get('todo')
    if todo:
        todos.append(todo)
    return redirect(url_for('index'))

@app.route('/delete/<int:todo_id>')
def delete_todo(todo_id):
    if 0 <= todo_id < len(todos):
        todos.pop(todo_id)
    return redirect(url_for('index'))
```

```
if __name__ == '__main__':  
    app.run(debug=True)
```

## Step 4: `templates/base.html` - Base Template

Create a base template `base.html`:

```
html  
Copy code  
<!DOCTYPE html>  
<html lang="en">  
<head>  
    <meta charset="UTF-8">  
    <meta name="viewport" content="width=device-width, initial-scale=1.0">  
    <title>Gryffindor To-Do App</title>  
    <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">  
</head>  
<body>  
    <nav class="navbar">  
        <div class="container">  
            <a href="/" class="navbar-brand">Gryffindor To-Do App</a>  
            <ul class="navbar-nav">  
                <li class="nav-item"><a href="/" class="nav-link">Home</a></li>  
            </ul>  
        </div>  
    </nav>  
    <div class="container">  
        {% block content %}{% endblock %}  
    </div>  
</body>  
</html>
```

## Step 5: `templates/index.html` - Index Template

Create the main content template `index.html`:

```
html  
Copy code  
{% extends 'base.html' %}  
  
{% block content %}  
<h1>To-Do List</h1>  
<form action="{{ url_for('add_todo') }}" method="POST">  
    <input type="text" name="todo" placeholder="Enter a new task">  
    <button type="submit">Add</button>  
</form>
```

```

<ul class="todo-list">
  {% for todo in todos %}
    <li>{{ todo }} <a href="{{ url_for('delete_todo', todo_id=loop.index0) }}">Delete</a></li>
  {% endfor %}
</ul>
{% endblock %}

```

### How It Works:

#### 1. Base Template (`base.html`):

- Defines the overall structure of the web pages.
- Includes common elements such as the navigation bar and container.
- Contains a `{% block content %}` which is a placeholder for content defined in other templates.

#### 2. Content Template (`index.html`):

- Extends `base.html`, meaning it uses the structure defined in `base.html`.
- Fills the `{% block content %}` with specific content for the main page.
- Includes a form for adding tasks and a list for displaying tasks.

When you navigate to the main page, Flask uses `index.html` which extends `base.html`. The content from `index.html` is inserted into the `{% block content %}` of `base.html`, resulting in a complete webpage with the Gryffindor theme, navigation bar, and to-do list functionality.

## Step 6: static/style.css - CSS Styles

Create a CSS file `style.css` for Gryffindor-themed styling:

```

css
Copy code
body {
  font-family: 'Arial', sans-serif;
  background-color: #ffe4b5;
  color: #4b2e83;
}

.navbar {
  background-color: #7f0909;
  color: #d3a625;
  padding: 10px;
}

```

```

.navbar-brand {
  color: #d3a625;
  font-size: 24px;
  text-decoration: none;
}

.navbar-nav {
  list-style: none;
  padding: 0;
}

.navbar-nav .nav-item {
  display: inline;
  margin-right: 10px;
}

.navbar-nav .nav-link {
  color: #d3a625;
  text-decoration: none;
}

.container {
  margin: 20px auto;
  max-width: 600px;
  text-align: center;
}

h1 {
  font-size: 36px;
  color: #7f0909;
}

form input[type="text"] {
  padding: 10px;
  font-size: 16px;
  width: 70%;
}

form button {
  padding: 10px 20px;
  font-size: 16px;
  background-color: #7f0909;
  color: #d3a625;
  border: none;
  cursor: pointer;
}

```

```
}

form button:hover {
  background-color: #d3a625;
  color: #7f0909;
}

.todo-list {
  list-style: none;
  padding: 0;
}

.todo-list li {
  background-color: #fff;
  padding: 10px;
  margin: 5px 0;
  border-radius: 4px;
  display: flex;
  justify-content: space-between;
}

.todo-list li a {
  color: #7f0909;
  text-decoration: none;
}
```

Folder structure

```
▼ FLASK
  > __pycache__
  ▼ static
    # style.css
  ▼ templates
    <> base.html
    <> index.html
  📄 app.py
  📄 hello.py
```

## Step 7: Run the Application

Now you can run your Flask application:

bash

Copy code

**python app.py**

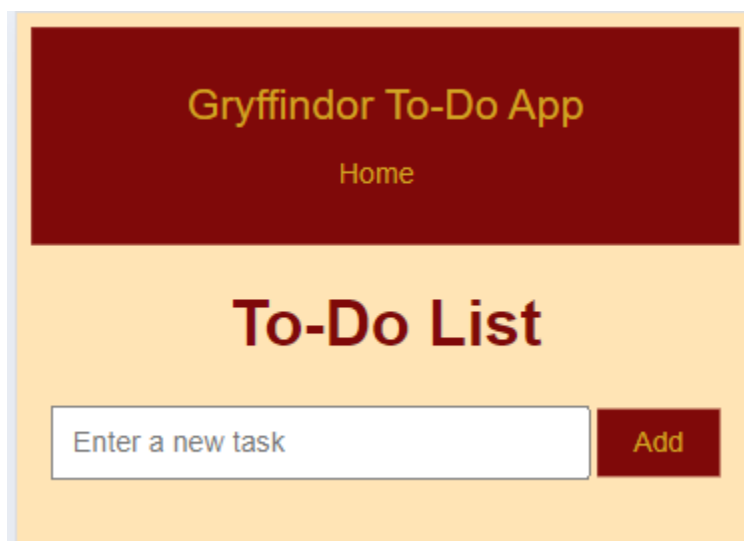
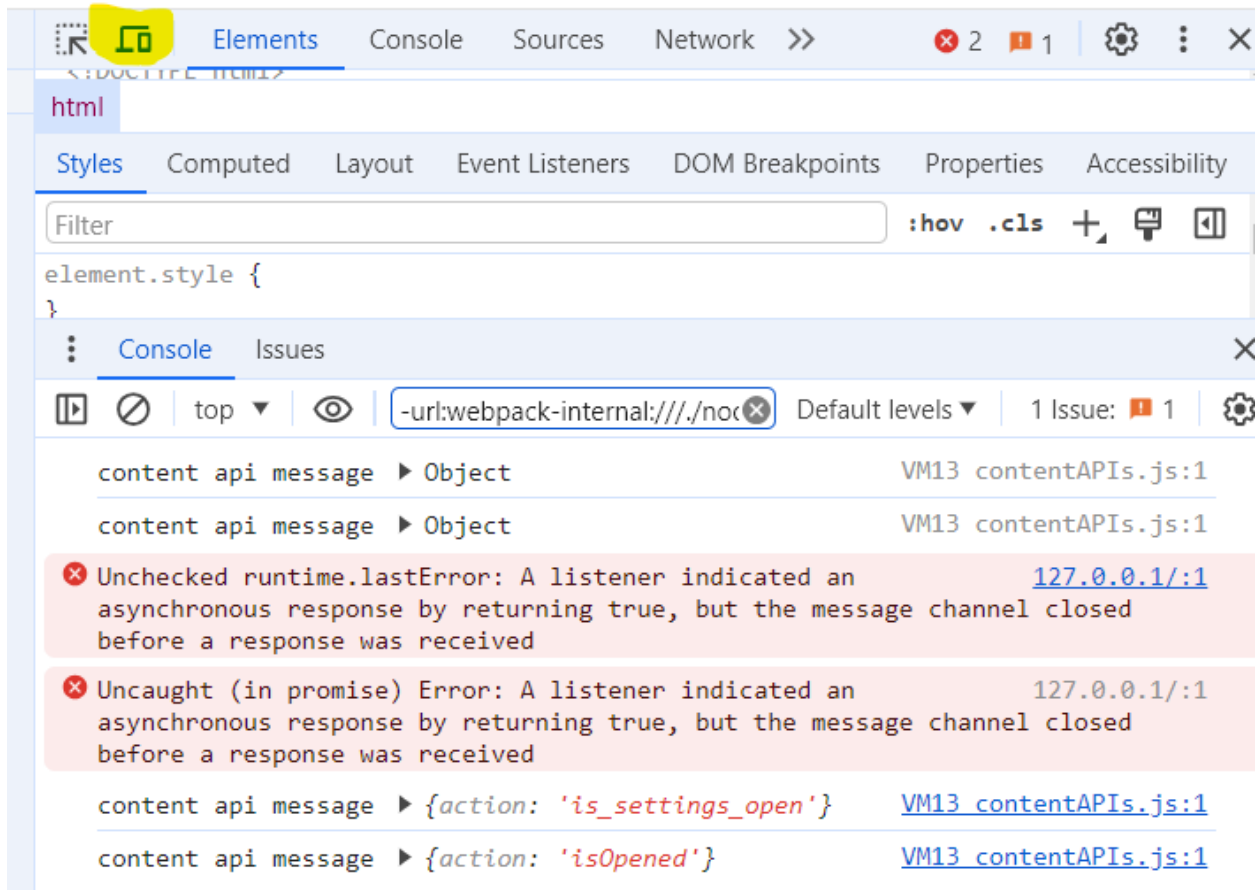
```
○ res\3320\flask> flask --app app run
  * Serving Flask app 'app'
  * Debug mode: off
  WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
  * Running on http://127.0.0.1:5000
  Press CTRL+C to quit
  □
```

Navigate to **http://127.0.0.1:5000** in your web browser, and you should see the Gryffindor-themed to-do app with a navbar and working functionality.

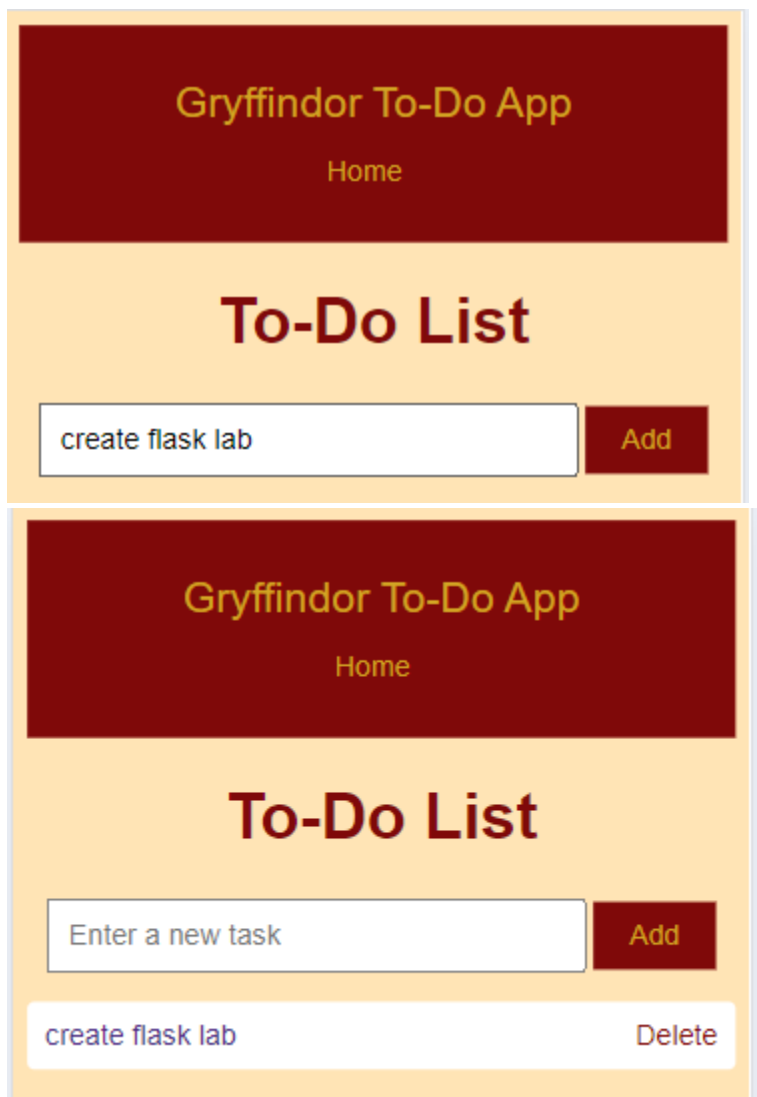
4o

**Right-click inspect -> go mobile mode**





Test the app



Gryffindor To-Do App

Home

To-Do List

Add

create flask lab

Delete

grade everything

Delete

Gryffindor To-Do App

Home

To-Do List

Add

grade everything

Delete

If there are errors on console you should fix them all

### 3. Verify with Gemini

<https://gemini.google.com/>

can you polish my app I want to make sure it is the latest version To create a Gryffindor-themed Flask to-do app with a navbar and working functionality, you can follow the steps below. This involves setting up Flask, creating the necessary HTML templates, and adding some CSS for the Gryffindor theme.

#### Step 1: Set Up Flask Environment

First, ensure you have Flask installed. You can install it using pip:



Output:

Absolutely! Here's the polished and updated version of your Gryffindor-themed Flask to-do app, incorporating best practices and addressing potential improvements:

Python

```
from flask import Flask, render_template, request, redirect, url_for
app = Flask(__name__)
# Store data in a list (consider using a database for persistence)
todos = []
@app.route('/')
def index():
    return render_template('index.html', todos=todos)
@app.route('/add', methods=['POST'])
def add_todo():
    todo = request.form.get('todo').strip() # Strip whitespace for cleaner input
    if todo:
        todos.append(todo)
    return redirect(url_for('index'))

@app.route('/delete/<int:todo_id>')
def delete_todo(todo_id):
    if 0 <= todo_id < len(todos):
        del todos[todo_id]
    return redirect(url_for('index'))
@app.route('/clear')
def clear_todos():
    todos.clear()
    return redirect(url_for('index'))
```

```
if __name__ == '__main__':  
    app.run(debug=True)
```

Use code with caution.

play\_circleeditcontent\_copy

Key Changes:

Whitespace Removal: Added `todo.strip()` to remove any leading or trailing whitespace from the user input.

Improved Deletion: Used `del todos[todo_id]` for cleaner and more efficient removal of items from the list.

Clear All Button: Added a new route `/clear` and a button in `index.html` to clear all tasks at once.

`templates/index.html` (updated):

HTML

```
{% extends 'base.html' %}
```

```
{% block content %}
```

```
    <h1>To-Do List</h1>
```

```
    <form action="{{ url_for('add_todo') }}" method="POST">
```

```
        <input type="text" name="todo" placeholder="Enter a new task">
```

```
        <button type="submit">Add</button>
```

```
    </form>
```

```
    <ul class="todo-list">
```

```
        {% for todo in todos %}
```

```
            <li>
```

```
                {{ todo }}
```

```
                <a href="{{ url_for('delete_todo', todo_id=loop.index0) }}">Delete</a>
```

```
            </li>
```

```
        {% endfor %}
```

```
    </ul>
```

```
    {% if todos %}
```

```
        <a href="{{ url_for('clear_todos') }}" class="clear-button">Clear All</a>
```

```
    {% endif %}
```

```
{% endblock %}
```

Use code with caution.

content\_copy

CSS (style.css) Improvements:

Font Family: Use a more Gryffindor-like font (e.g., "Cinzel Decorative" or "Lora").

Background Texture: Consider adding a subtle parchment-like background image.

More Thematic Colors: Use Gryffindor colors (red, gold) more prominently throughout the design.

Responsiveness: Make sure the layout adapts well to different screen sizes.

Additional Tips:

**Database Integration: Replace the in-memory todos list with a database (e.g., SQLite) for persistent storage.**

User Authentication: If you want to track tasks for individual users, add user registration and login features.

Task Prioritization/Categories: Allow users to prioritize tasks or organize them into categories.

Date/Time Reminders: Implement reminders based on dates and times.

Let me know if you have any other questions or need further refinements!

Sources

info

**[github.com/gp48maz1/to-do-test](https://github.com/gp48maz1/to-do-test)**

**Result**

```
res\3320\flask> flask --app app run
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
□
```

Gryffindor To-Do App

Home

To-Do List

Add

grade everything

Delete

Gryffindor To-Do App

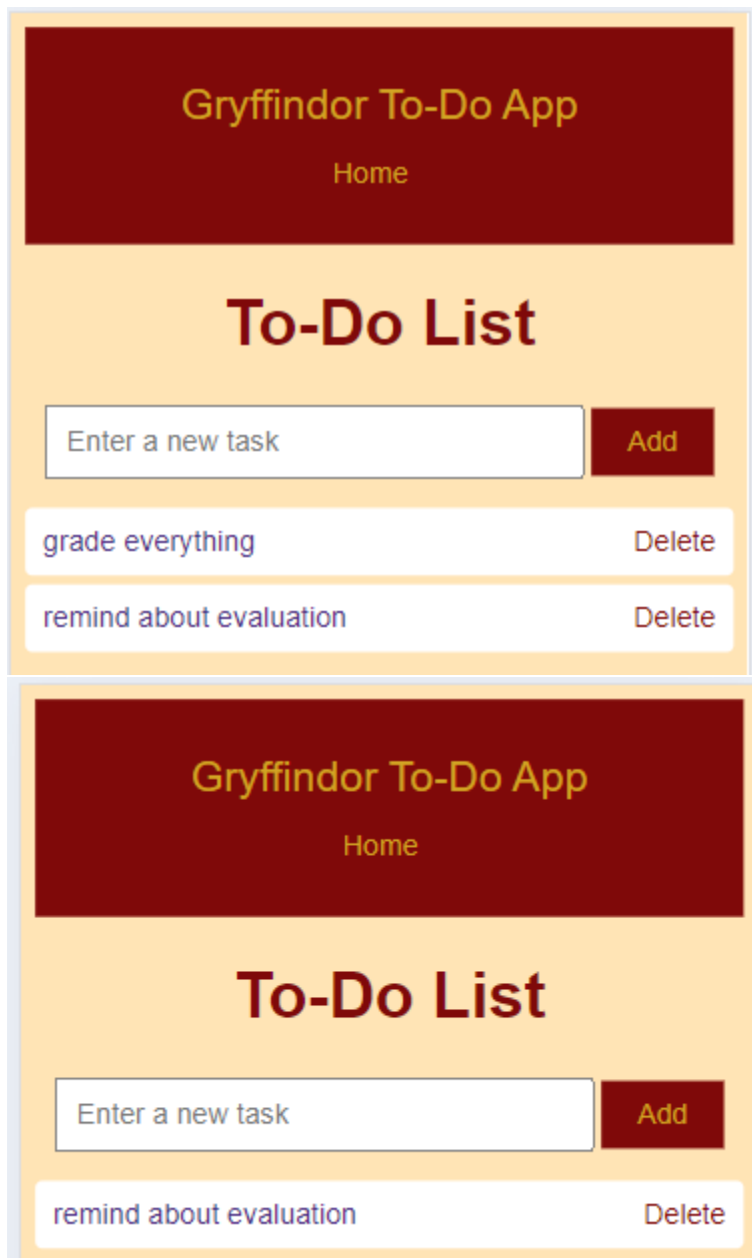
Home

To-Do List

Add

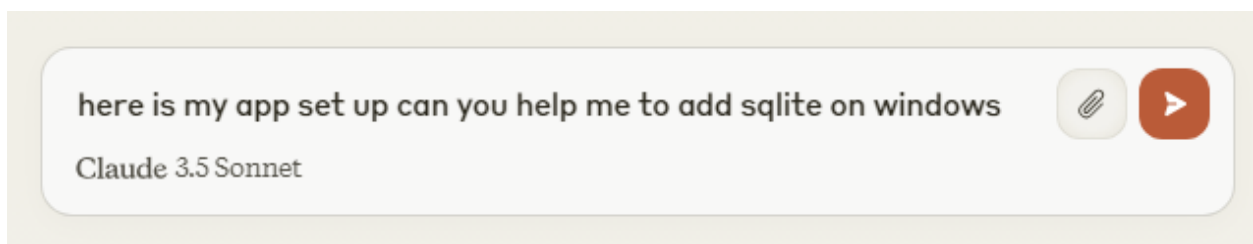
grade everything

Delete



4. Add some Claude

<https://www.anthropic.com/claude>



**You code listed**



```
<ul class="todo-list">
  {% for todo in todos %}
    <li>
      {{ todo }}
      <a href="{{ url_for('delete_todo',
todo_id=loop.index0) }}">Delete</a>
    </li>
  {% endfor %}
</ul>

{% if todos %}
  <a href="{{ url_for('clear_todos') }}" class="clear-
button">Clear All</a>
{% endif %}
{% endblock %}
```

Claude 3.5 Sonnet

⚠️ Claude does not have internet access and cannot get up to date info from web links.

Output:

Certainly! I'd be happy to help you add SQLite support to your Flask application on Windows. Here's how you can modify your existing code to use SQLite:

First, install the required package:

Open a command prompt and run:

Copy

**pip install flask-sqlalchemy**

```
\3320\flask> pip3 install flask-sqlalchemy
```

Now, let's modify your app.py file:

pythonCopy

```

from flask import Flask, render_template, request, redirect, url_for
from flask_sqlalchemy import SQLAlchemy
import os

app = Flask(__name__)

# Configure SQLite database
basedir = os.path.abspath(os.path.dirname(__file__))
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:/// ' + os.path.join(basedir,
'todos.db')
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False

db = SQLAlchemy(app)

# Define Todo model
class Todo(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    content = db.Column(db.String(200), nullable=False)

    def __repr__(self):
        return f'<Todo {self.id}>'

# Create tables
with app.app_context():
    db.create_all()

@app.route('/')
def index():
    todos = Todo.query.all()
    return render_template('index.html', todos=todos)

@app.route('/add', methods=['POST'])
def add_todo():
    todo_content = request.form.get('todo').strip()
    if todo_content:
        new_todo = Todo(content=todo_content)
        db.session.add(new_todo)
        db.session.commit()
    return redirect(url_for('index'))

@app.route('/delete/<int:todo_id>')
def delete_todo(todo_id):

```

```

    todo = Todo.query.get_or_404(todo_id)
    db.session.delete(todo)
    db.session.commit()
    return redirect(url_for('index'))

```

```

@app.route('/clear')
def clear_todos():
    Todo.query.delete()
    db.session.commit()
    return redirect(url_for('index'))

```

```

if __name__ == '__main__':
    app.run(debug=True)

```

Update your templates/index.html file:

htmlCopy

```

{% extends 'base.html' %}

{% block content %}
    <h1>To-Do List</h1>

    <form action="{{ url_for('add_todo') }}" method="POST">
        <input type="text" name="todo" placeholder="Enter a new task">
        <button type="submit">Add</button>
    </form>

    <ul class="todo-list">
        {% for todo in todos %}
            <li>
                {{ todo.content }}
                <a href="{{ url_for('delete_todo', todo_id=todo.id) }}">Delete</a>
            </li>
        {% endfor %}
    </ul>

    {% if todos %}
        <a href="{{ url_for('clear_todos') }}" class="clear-button">Clear All</a>
    {% endif %}
{% endblock %}

```

Key changes:

We've imported SQLAlchemy and set up the database configuration.

A Todo model is defined to represent our database table.

We use `db.create_all()` to create the table if it doesn't exist.

CRUD operations now interact with the database instead of a list.

In the template, we access `todo.content` instead of just `todo`.

This setup will create a SQLite database file named `todos.db` in your project directory. The data will persist between application restarts.

Would you like me to explain any part of this code in more detail?



I apologize for the confusion. It seems the error is occurring because the module 'flask\_sqlalchemy' is not installed or not found in your Python environment. Let's troubleshoot this issue:

First, make sure you've installed Flask-SQLAlchemy correctly. Open a command prompt or terminal and run:

Copy **pip install Flask-SQLAlchemy**

```
● res\3320\flask> pip3 install Flask-SQLAlchemy
```

If that doesn't work, try specifying the version:

Copy

**pip install Flask-SQLAlchemy==2.5.1**

If you're using a virtual environment (which is recommended), make sure you've activated it before installing the package and running your Flask app.

After installation, try importing it in your Python interpreter to check if it's recognized:

```
pythonCopy
```

```
from flask_sqlalchemy import SQLAlchemy
```

If you're still having issues, check your Python path and make sure the installation directory is included.

As a last resort, you could try uninstalling and reinstalling both Flask and Flask-SQLAlchemy:

```
Copy
```

```
pip uninstall flask flask-sqlalchemy
```

```
pip install flask flask-sqlalchemy
```

If none of the above works, there might be an issue with your Python installation or environment setup. In this case, you might want to consider setting up a new virtual environment.

Once you've successfully installed Flask-SQLAlchemy, your import statement should work:

```
pythonCopy
```

```
from flask_sqlalchemy import SQLAlchemy
```

If you continue to face issues, please provide more details about your Python environment, such as the Python version you're using and whether you're using any virtual environment tools. This information will help in providing a more specific solution.

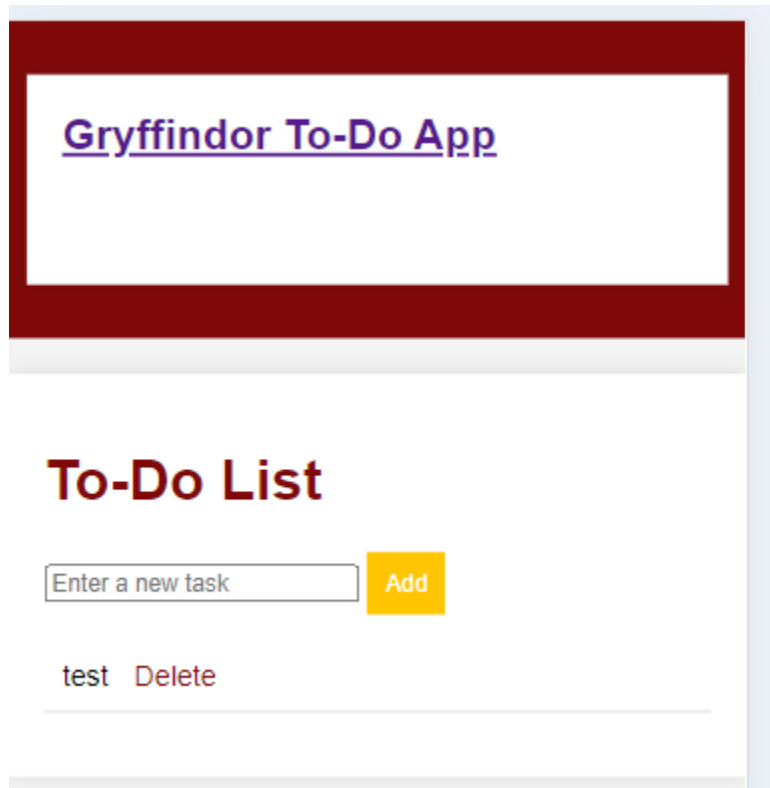
⚠️ Claude does not have internet access and cannot get up to date info from web links.

I do not have todos.db in my project directory

Claude 3.5 Sonnet

Use **shift + return** for new line

## 5. ChatGPT to the rescue



It looks like you encountered an error related to using SQLAlchemy outside of an application context. To resolve this, you need to ensure that the application context is set up correctly before calling `db.create_all()`.

Here's the updated version of your Flask application with the appropriate context setup:

1. Ensure you have a directory structure like this:

```
csharp
Copy code
project/
├── app.py
├── templates/
│   ├── base.html
│   └── index.html
└── static/
    └── style.css
```

2. Update your `app.py` file as follows:

```
python
Copy code
from flask import Flask, render_template, request, redirect, url_for
from flask_sqlalchemy import SQLAlchemy

app = Flask(__name__)
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///todos.db'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
db = SQLAlchemy(app)

class Todo(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    task = db.Column(db.String(200), nullable=False)

with app.app_context():
    db.create_all()

@app.route('/')
def index():
    todos = Todo.query.all()
    return render_template('index.html', todos=todos)

@app.route('/add', methods=['POST'])
def add_todo():
    task = request.form.get('todo')
    new_todo = Todo(task=task)
    db.session.add(new_todo)
    db.session.commit()
    return redirect(url_for('index'))

@app.route('/delete/<int:todo_id>')
def delete_todo(todo_id):
    todo = Todo.query.get(todo_id)
    db.session.delete(todo)
    db.session.commit()
    return redirect(url_for('index'))

if __name__ == "__main__":
    app.run(debug=True)
```

### 3. Create the templates/base.html file:

```
html
Copy code
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Gryffindor To-Do App</title>
    <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
</head>
<body>
    <nav class="navbar">
        <div class="container">
```

```

        <a href="/" class="navbar-brand">Gryffindor To-Do App</a>
        <ul class="navbar-nav">
            <li class="nav-item"><a href="/" class="nav-
link">Home</a></li>
        </ul>
    </div>
</nav>
<div class="container">
    {% block content %}{% endblock %}
</div>
</body>
</html>

```

#### 4. Create the templates/index.html file:

```

html
Copy code
{% extends 'base.html' %}

{% block content %}
<h1>To-Do List</h1>
<form action="{{ url_for('add_todo') }}" method="POST">
    <input type="text" name="todo" placeholder="Enter a new task">
    <button type="submit">Add</button>
</form>
<ul class="todo-list">
    {% for todo in todos %}
        <li>{{ todo.task }} <a href="{{ url_for('delete_todo', todo_id=todo.id)
}}">Delete</a></li>
    {% endfor %}
</ul>
{% endblock %}

```

#### 5. Create the static/style.css file:

```

css
Copy code
body {
    font-family: Arial, sans-serif;
    background-color: #f4f4f4;
    margin: 0;
    padding: 0;
}

.navbar {
    background-color: #7f0909; /* Gryffindor red */
    color: white;
    padding: 10px;
}

.navbar .navbar-brand {
    font-size: 24px;
    font-weight: bold;
}

```



```

.navbar .nav-link {
  color: white;
  text-decoration: none;
  margin-left: 15px;
}

.container {
  max-width: 600px;
  margin: 20px auto;
  padding: 20px;
  background-color: white;
  box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
}

h1 {
  color: #7f0909; /* Gryffindor red */
}

button {
  background-color: #ffc500; /* Gryffindor gold */
  color: white;
  border: none;
  padding: 10px;
  cursor: pointer;
}

button:hover {
  background-color: #e0b000;
}

.todo-list {
  list-style: none;
  padding: 0;
}

.todo-list li {
  padding: 10px;
  border-bottom: 1px solid #ddd;
}

.todo-list a {
  color: #7f0909; /* Gryffindor red */
  text-decoration: none;
  margin-left: 10px;
}

.todo-list a:hover {
  text-decoration: underline;
}

```

## 6. Run your Flask application:

```

shell
Copy code
python app.py

```

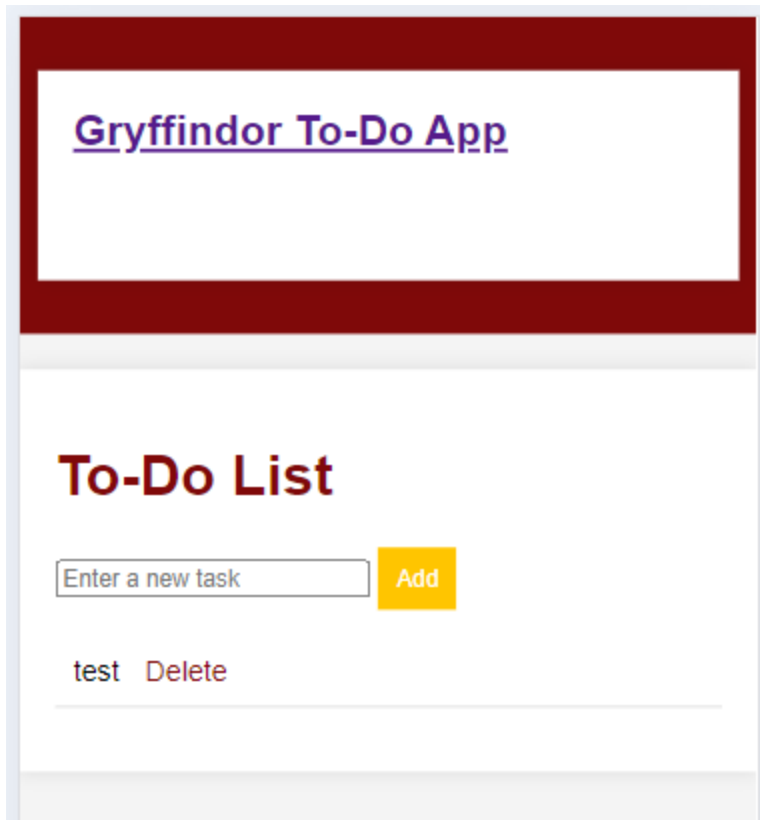
With these updates, you should have a working Gryffindor-themed Flask to-do app with a navbar and the required functionality.

**ymment. Use a production WSGI server instead.**

```
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 136-361-734
127.0.0.1 - - [24/Jun/2024 09:30:07] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [24/Jun/2024 09:30:07] "GET /static/style.css HTTP/1.1" 200
-
127.0.0.1 - - [24/Jun/2024 09:30:10] "GET /static/style.css HTTP/1.1" 304
-
127.0.0.1 - - [24/Jun/2024 09:30:16] "POST /add HTTP/1.1" 302 -
127.0.0.1 - - [24/Jun/2024 09:30:16] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [24/Jun/2024 09:30:16] "GET /static/style.css HTTP/1.1" 304
-
□
```

Quit and Veriy that the test will be there after app restart

```
127.0.0.1 - - [24/Jun/2024 09:30:16] "POST /add HTTP/1.1" 302 -
127.0.0.1 - - [24/Jun/2024 09:30:16] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [24/Jun/2024 09:30:16] "GET /static/style.css HTTP/1.1" 304
-
○ (myenv) PS C:\Users\galae\Desktop\00472-337513824 GS Backup 12.20.22 JC\KE
AN_Lectures\3320\flask> python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deplo
ymment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 136-361-734
□
```



Follow the LLM suggestions but we aware of every step. Hint:

First you need to set up this

```
gryffindor_todo_app/  
├── app.py  
├── templates/  
│   ├── base.html  
│   └── index.html  
├── static/  
└── style.css
```

Standard Set Up (if LLMS do not work for you): follow  
<https://www.youtube.com/watch?v=3vfum74ggHE>

```
python3 -m venv venv  
.\venv\Scripts\activate  
pip install Flask Flask-SQLAlchemy
```

Set up the Flask application

Create an app.py file under the Flask folder

Source code

```
from flask import Flask, render_template, request, redirect, url_for
from flask_sqlalchemy import SQLAlchemy
```

```
app = Flask(__name__)
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///db.sqlite'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
db = SQLAlchemy(app)
```

```
class Todo(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    title = db.Column(db.String(100))
    complete = db.Column(db.Boolean)
```

```
db.create_all()
```

```
@app.route("/")
def home():
    todo_list = db.session.query(Todo).all()
    return render_template("base.html", todo_list=todo_list)
```

```
@app.route("/add", methods=["POST"])
def add():
    title = request.form.get("title")
    new_todo = Todo(title=title, complete=False)
    db.session.add(new_todo)
    db.session.commit()
    return redirect(url_for("home"))
```

```
@app.route("/update/<int:todo_id>")
def update(todo_id):
    todo = db.session.query(Todo).filter_by(id=todo_id).first()
    todo.complete = not todo.complete
    db.session.commit()
    return redirect(url_for("home"))
```

```
@app.route("/delete/<int:todo_id>")
def delete(todo_id):
```

```

    todo = db.session.query(Todo).filter_by(id=todo_id).first()
    db.session.delete(todo)
    db.session.commit()
    return redirect(url_for("home"))

```

```

if __name__ == "__main__":
    app.run(debug=True)

```

## HTML template (base.html):

Create a folder named templates and a file base.html in it

```

<!DOCTYPE html>
<html>
<head>
    <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/semantic-
ui@2.4.2/dist/semantic.min.css">
</head>
<body>
    <div style="margin-top: 50px;" class="ui container">
        <h1 class="ui center aligned header">To Do App</h1>
        <form class="ui form" action="/add" method="post">
            <div class="field">
                <label>Todo Item</label>
                <input type="text" name="title" placeholder="Enter Todo...">
            </div>
            <button class="ui blue button" type="submit">Add</button>
        </form>
        <div class="ui segment">
            {% for todo in todo_list %}
            <div class="ui segment">
                <h2 class="ui header">{{ todo.title }}</h2>
                <div class="ui buttons">
                    <a class="ui blue button" href="/update/{{ todo.id }}">Update</a>
                    <a class="ui red button" href="/delete/{{ todo.id }}">Delete</a>
                </div>
            </div>
            {% endfor %}
        </div>
    </div>

```

```
</body>
</html>
```

## Set up environmental variables

Navigate to your folder

Set the environment variables:

For Command Prompt:

```
cmd
```

```
cmd
```

```
set FLASK_APP=app.py
```

```
set FLASK_ENV=development
```

For PowerShell:

```
powershell
```

```
$env:FLASK_APP = "app.py"
```

```
$env:FLASK_ENV = "development"
```

## Export environmental variables

```
export FLASK_APP=app.py
```

```
export FLASK_ENV=development
```

```
from flask import Flask, render_template, request, redirect, url_for
from flask_sqlalchemy import SQLAlchemy
```

```
app = Flask(__name__)
```

```
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///db.sqlite'
```

```
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
```

```
db = SQLAlchemy(app)
```

```
class Todo(db.Model):
```

```
    id = db.Column(db.Integer, primary_key=True)
```

```
    title = db.Column(db.String(100))
```

```
    complete = db.Column(db.Boolean)
```

```
db.create_all()
```

```
@app.route("/")
```

```
def home():
```

```
    todo_list = db.session.query(Todo).all()
```

```
    return render_template("base.html", todo_list=todo_list)
```

```
@app.route("/add", methods=["POST"])
```

```

def add():
    title = request.form.get("title")
    new_todo = Todo(title=title, complete=False)
    db.session.add(new_todo)
    db.session.commit()
    return redirect(url_for("home"))

@app.route("/update/<int:todo_id>")
def update(todo_id):
    todo = db.session.query(Todo).filter_by(id=todo_id).first()
    todo.complete = not todo.complete
    db.session.commit()
    return redirect(url_for("home"))

@app.route("/delete/<int:todo_id>")
def delete(todo_id):
    todo = db.session.query(Todo).filter_by(id=todo_id).first()
    db.session.delete(todo)
    db.session.commit()
    return redirect(url_for("home"))

if __name__ == "__main__":
    app.run(debug=True)

```

Run the app

pip install flask\_sqlalchemy

flask run